

1. Introduction

Un graphe est défini par deux ensembles : Un ensemble de sommets noté X et un ensemble de relations entre les sommets noté U ou E . Selon le type de cette relation.

Mais comment la notion de graphe est apparue ? Quant ?

Le problème est apparu dans la ville des sept ponts de Königsberg, où ses habitants cassent leurs têtes à traverser les sept ponts en une seule et unique fois?

Mais personne n'a réussi! Est-il possible de le faire?

C'est là où Euler est venu pour résoudre ce problème en 18^{ème} siècle (1736) en formalisant le problème, ainsi il a dessiné un graphe où il a utilisé les mathématiques pour le prouver.

Euler est sorti avec un théorème qui est « Théorème d'Euler », les années suivantes d'autres savants sont venus pour continuer et approfondir les études sur les graphes, les arbres, les listes....

On distingue deux grandes classes de graphes : les graphes orientés si la relation est orientée et les graphes non orientés dans le cas contraire, pour plus de précision on a défini des notions dans ce qui suit.

2. Objectifs du travail

Notre objectif est bien de trouver la meilleure solution du graphe (matrice d'adjacence ou bien liste d'adjacence)

D'abord nous allons faire des recherches sur les graphes, puis on écrira quelques opérations sur les deux méthodes, matrice d'adjacence et liste d'adjacence. Après nous allons faire l'évaluation et la comparaison des complexités théoriques et expérimentales pour découvrir les différences entre elles et définir la meilleure méthode

3. Partie I : Les définitions

Un graphe :

Un graphe est une structure composée d'objets dans laquelle certaines paires d'objets sont en relation. Un graphe a un ensemble fini de sommets reliés par des arêtes, on distingue les graphes non orientés et les graphes orientés.

Graphe orienté :

Les arêtes relient deux sommets de manière symétrique, de sorte que les arêtes possèdent une orientation dans un sens restreint mais très répandu du terme.

Graphe non orienté :

Un graphe non orienté est un couple formé d'un ensemble de sommets et un ensemble d'arêtes, chaque arête étant une paire de sommets.

Cette définition ne s'applique qu'aux graphes simples et n'est pas valable pour les multi-graphes.

Arrête :

C'est une ligne aussi appelée *flèche*, relie deux sommets consécutifs, distincts ou non de manière asymétrique, d'un graphe non orienté.

Arc :

Un arc est une ligne représentée par une flèche ou une arête orientée, elle joint deux sommets consécutifs, distincts ou non, d'un graphe orienté.

Cycle :

Dans un graphe orienté, un cycle est une suite d'arêtes consécutives (chaîne simple) dont les deux sommets extrémités sont identiques.

Circuit :

Dans un graphe orienté, on appelle circuit une suite d'arcs consécutifs (chemin) dont les deux sommets extrémités sont identiques.

Graphe pondéré :

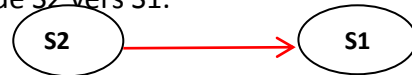
Un graphe pondéré est un graphe étiqueté (étiquette peut-être un : mot, lettre, symbole, ...) dont toutes les étiquettes sont des nombres réels positifs ou nuls. Ces nombres sont les poids des liaisons (arêtes ou arcs) entre les sommets.

Degré d'un nœud :

Le degré d'un nœud est la quantité d'arêtes qui lui sont incidentes. Les arêtes incidentes sur un nœud sont des arêtes qui relient ce nœud à un autre (ou à lui-même, dans le cas d'une boucle ou à des arêtes qui touchent ce nœud).

Arc entrant :

Un arc est une ligne représentée par une flèche orientée d'un sommet 1 (S1) vers un sommet 2 (S2), en dit que S1 a un arc entrant si l'arc va de S2 vers S1.



Arc sortant :

Un arc est une ligne représentée par une flèche orientée d'un sommet 1 (S1) vers un sommet 2 (S2), en dit que S1 a un arc sortant si l'arc va de S1 vers S2.



Graphe simple :

Est un graphe sans boucle et sans arête multiple. Chaque paire de sommets est reliée par au plus une arête

Multi-graphe:

Contrairement au graphe simple, le multi-graphe est un graphe qui autorise des liens multiples (liens parallèles). En d'autres termes, dans un multi-graphe, deux sommets peuvent être connectés par plus d'un lien. Multi-graphe est dit orienté s'il autorise des liens orientés.

Densité d'un graphe :

La densité d'un graphe désigne le rapport entre le nombre de liens présents et le nombre de liens possibles ; elle varie entre 0 (graphe vide) et 1 (graphe complet). Si la formule change en fonction du type de graphe, elle reste aisée à déterminer et à interpréter.

Graphe connexe :

Un graphe est un graphe en un seul morceau, il est dit aussi connexe quand tout sommet peut être relié à tout autre sommet par une arête ou une suite d'arêtes.

Composante connexe :

Dans un graphe non orienté, une composante connexe est un sous-graphe induit maximal connexe, autrement dit c'est un ensemble de points qui sont reliés deux à deux par un chemin. On peut ainsi regrouper les sommets d'un graphe selon leur appartenance à la même composante connexe.

Profondeur :

Le parcours en profondeur d'un graphe à partir d'un sommet consiste à suivre les arêtes arbitrairement, en marquant les sommets déjà visités pour ne pas les visiter à nouveau.

Graphe complet :

Un graphe complet est un graphe simple dont tous les sommets sont adjacents deux à deux, c'est-à-dire que tout couple de sommets disjoints est relié par une arête. Si le graphe est orienté, on dit qu'il est complet si chaque paire de sommets est reliée par exactement deux arcs (un dans chaque sens).

4. Partie II : Les représentations d'un graphe en mémoire

Représentation des graphes par deux méthodes possible (matrice d'adjacence, liste d'adjacence) :

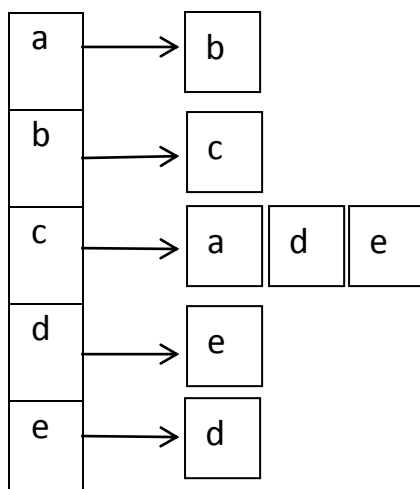
Graphe G1 :

Représentation par matrice adjacence :

Le graphe est simple et orienté

	a	b	c	d	e
a	0	1	0	0	0
b	0	0	1	0	0
c	1	0	0	1	1
d	0	0	0	0	1
e	0	0	0	1	0

Représentation par liste d'adjacence :



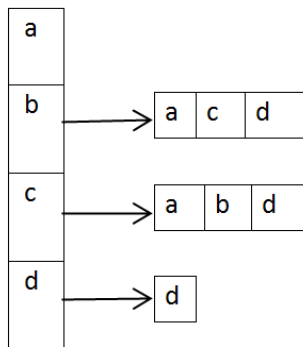
Graphe G2 :

Représentation par matrice adjacence :

Le graphe n'est pas simple car il contient une boucle dans le « d » et il est orienté

	A	b	c	d
a	0	0	0	0
b	1	0	1	1
c	1	1	0	1
d	0	0	0	1

Représentation par liste d'adjacence :



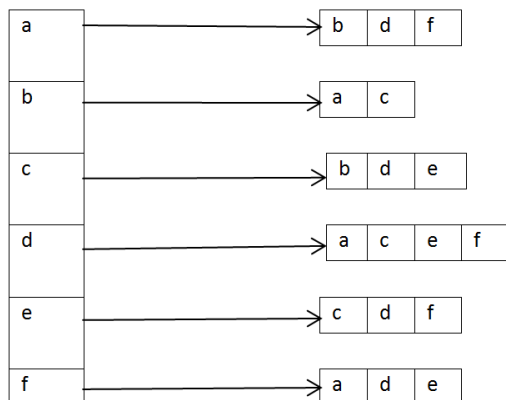
Graphe G3 :

Représentation par matrice adjacence :

Le graphe est simple et non orienté

	a	b	c	d	e	f
a	0	1	0	1	0	1
b	1	0	1	0	0	0
c	0	1	0	1	1	0
d	1	0	1	0	1	1
e	0	0	1	1	0	1
f	1	0	0	1	1	0

Représentation par liste d'adjacence :



5. Partie III : Opérations et complexité

Description des différents algorithmes (avec commentaires) et calcul de complexité :

I. LISTE D'ADJACENCE

1. Construction d'un graphe orienté/non orienté

```
// Structure de nœud pour représenter un nœud du graphe
typedef struct sommet {
    int val;
    struct sommet *suiv;
} Sommet;

// Tableau de tête de liste pour représenter le graphe
Sommet *liste_adja[MAX_SOMMETS];
int num_sommets;

// Ajouter un nœud au graphe
void add_sommet(int val) {
    // Allouer la mémoire pour le nouveau nœud
    Sommet *nv_sommet = malloc(sizeof(Sommet));
    nv_sommet->val = val;
    nv_sommet->suiv = NULL;

    // Ajouter le nœud à la liste d'adjacence
    liste_adja[num_sommets++] = nv_sommet;
}
```

Complexité :

La complexité de la fonction "add_sommet" est $O(1)$, c'est une complexité constante. La fonction effectue un nombre fixe d'opérations (allouer de la mémoire pour un nouveau nœud, initialiser les valeurs du nœud, ajouter le nœud à la liste d'adjacence)

2. Affichage du graphe (représentation mémoire).

```
// Représentation du graphe
void rep_graph(void) {
    for (int i = 0; i < num_sommets; i++) {
        Sommet *cur = liste_adja[i];
        printf("%d :", cur->val); // pour afficher le sommet
        cur = cur->suiv;
        while (cur != NULL) {
            printf(" %d", cur->val);
            cur = cur->suiv;
        }
        printf("\n");
    }
}
```

Complexité :

la complexité de la fonction "rep_graph" est de $O(m*n)$ où m est le nombre d'arêtes et n est le nombre de sommets dans le graphe.

3. Calculer la densité du graphe

```
// Calculer la densité du graphe
double densite(void) {
    int num_liens = 0;
    for (int i = 0; i < num_sommets; i++) {
        Sommet *cur = liste_adja[i];

        while (cur->suiv != NULL) {
            num_liens++;
            cur = cur->suiv;
        }
    }
    return (2.0 * num_liens) / (num_sommets * (num_sommets - 1));
}
```

Complexité :

la complexité est de $O(m*n)$ où m est le nombre d'arêtes et n est le nombre de sommets dans le graphe.

4. Recherche d'un nœud a dans le graphe (afficher le nœud et ses liens)

```
// Rechercher un nœud dans le graph
bool rech_sommet(int val) {
    for (int i = 0; i < num_sommets; i++) {
        Sommet *cur = liste_adja[i];
        if (cur->val == val) {
            return true;
        }
    }
    return false;
}
```

Complexité :

La complexité de la fonction "rech_sommet" est $O(n)$ où n est le nombre de sommets dans le graphe.

5. Recherche d'un chemin entre un nœud a et un nœud b

```
// Rechercher un chemin entre deux nœuds dans le graph
bool DFS(int start, int end, bool visited[])
{
    if (start == end)
        return true;

    // On marque le sommet actuel comme visité et l'affiche
    visited[start] = true;
    Sommet* i = liste_adja[start];
    while(i)
    {
        if (!visited[i->val])
            if (DFS(i->val, end, visited))
                return true;
        i = i->suiv;
    }
    return false;
}
```

Complexité :

$O(n+m)$ où n est le nombre de sommets et m est le nombre d'arêtes dans le graphe. La fonction utilise une recherche en profondeur pour trouver un chemin entre deux nœuds, ce qui signifie qu'elle parcourt tous les nœuds accessibles à partir du nœud de départ.

6. Recherche du chemin le plus court entre deux nœuds a et b

```
// Recherche du chemin le plus court entre deux nœuds a et b
void bfs(int start, int end) {
    bool visited[num_sommets];
    int parent[num_sommets];
    int distance[num_sommets];
    int queue[num_sommets];
    int front = 0, rear = -1;
    // initialize all the arrays
    for (int i = 0; i < num_sommets; i++) {
        visited[i] = false;
        parent[i] = -1;
        distance[i] = INT_MAX;
    }

    // on met la distance = 0
    distance[start] = 0;
    visited[start] = true;
    queue[++rear] = start;
    // On utilise la methode BFS
    while (front <= rear) {
        int current = queue[front++];
        for (Sommet* adj = liste_adja[current]; adj != NULL; adj = adj->suiv) {
            if (!visited[adj->val]) {
                visited[adj->val] = true;
                parent[adj->val] = current;
                distance[adj->val] = distance[current] + 1;
                queue[++rear] = adj->val;
            }
        }
    }

    // On affiche le plus court chemin
    if (parent[end] == -1) {
        printf("Y a pas de chemin de %d a %d\n", start, end);
    } else {
        int current = end;
        int path[num_sommets];
        int path_index = 0;
        while (current != -1) {
            path[path_index++] = current;
            current = parent[current];
        }
        printf("Le plus court chemin entre %d et %d a une longueur de %d: ", start, end, distance[end]);
        for (int i = path_index - 1; i >= 0; i--) {
            printf("%d ", path[i]);
        }
    }
}
```

Complexité :

La complexité est $O(n+m)$ où n est le nombre de sommets et m est le nombre d'arêtes dans le graphe. La fonction utilise une recherche en largeur pour trouver le chemin le plus court entre deux nœuds, ce qui signifie qu'elle parcourt tous les nœuds accessibles à partir du nœud de départ.

8. Ajouter un nœud a avec ses liens

```
// Ajouter un nœud avec ses liens au graph
void add_sommet_avec_liens(int val) {
    // On fait l'appel a la fonction qui ajoute le sommet au graph
    add_sommet(val);
    int b, x, y;
    printf("Combien de liens ?\n");
    scanf("%d", &y);
    for (int i = 1; i < y+1; i++) {
        printf("Ecrit lien %d\n", i);
        scanf("%d", &b);
        printf("Ecrit 1 si c'est un lien orienté\nEcrit 0 sinon.\n");
        scanf("%d", &x);
        if (x == 0) {add_lien_NO(val, b);}
        else if (x == 1) {add_lien(val, b);}
    }
}
```

Complexité :

La complexité est $O(1+m)$ où m est le nombre d'arêtes. La fonction appelle d'abord la fonction "add_sommet" pour ajouter un nouveau sommet au graphe, ce qui a une complexité $O(1)$. Et elle fait l'appel à la fonction "add_lien" qui est $O(m)$ car cette fonction parcourt tous les liens de la liste d'adjacence liés au sommet "from" pour trouver le dernier sommet lié à celui-ci et ajoute un nouveau lien à la fin de la liste d'adjacence.

9. Supprimer un nœud a avec ses liens

```
// Supprimer un nœud avec ses liens
void supp_sommet_avec_liens(int val) {
    // supprimer tous les liens
    for (int i = 0; i < num_sommets; i++) {
        Sommet *cur = liste_adja[i], *prec = NULL;
        while (cur != NULL) {
            if (cur->val == val) {
                if (prec)
                    prec->suiv = cur->suiv;
                else
                    liste_adja[i] = cur->suiv;
                free(cur);
                cur = prec ? prec->suiv : liste_adja[i];
            }
            else {
                prec = cur;
                cur = cur->suiv;
            }
        }
    }
    //supprimer le sommet du graphe
    for (int i = 0; i < num_sommets; i++) {
        if (liste_adja[i]->val == val) {
            Sommet* temp = liste_adja[i];
            for (int j = i; j < num_sommets-1; j++) {
                liste_adja[j] = liste_adja[j+1];
            }
            free(temp);
            num_sommets--;
            break;
        }
    }
}
```

Complexité :

La complexité est $O(n*m)$. La fonction parcourt tous les sommets de la liste d'adjacence deux fois pour supprimer un sommet donné et tous ses liens avec les autres sommets.

10. Ajouter un lien (arc ou arête) entre deux nœuds existants

```
// Ajouter une arête au graph
void add_lien_NO(int from, int to) {
    // Allouer de la mémoire pour le nouveau nœud
    Sommet *nv_sommet = malloc(sizeof(Sommet));
    nv_sommet->val = to;
    nv_sommet->suiv = NULL;

    // Ajouter le nœud à la fin de la liste d'adjacence du nœud "from"
    Sommet *cur = liste_adja[from];
    while (cur->suiv != NULL) {
        cur = cur->suiv;
    }
    cur->suiv = nv_sommet;

    nv_sommet = malloc(sizeof(Sommet));
    nv_sommet->val = from;
    nv_sommet->suiv = NULL;
    // Dans le cas ou le lien n'est pas orienté on ajoute ça
    // Ajouter le nœud à la fin de la liste d'adjacence du nœud "to"
    cur = liste_adja[to];
    while (cur->suiv != NULL) {
        cur = cur->suiv;
    }
    cur->suiv = nv_sommet;
}
```

Complexité :

la complexité de la fonction "add_lien_NO" est $O(m^2)$ où m est le nombre d'arêtes dans le graphe, car pour chaque arête ajoutée, la fonction parcourt toutes les listes d'adjacences de deux sommets, une fois pour ajouter l'arête à partir du sommet "from" et une fois pour ajouter l'arête à partir du sommet "to".

II. MATRICE D'ADJACENCE

1. Construction d'un graphe orienté/non orienté

```
// Matrice d'adjacence pour représenter un graph
int matrice_adj[MAX_SOMMETS][MAX_SOMMETS];
int num_sommets;

// Initialiser le graph
void construct_graph(int num_sommets) {
    for (int i = 0; i < num_sommets; i++) {
        for (int j = 0; j < num_sommets; j++) {
            matrice_adj[i][j] = 0;
        }
    }
}
```

Complexité :

La complexité est $O(n^2)$ où n est le nombre de sommets dans le graphe.

2. Affichage du graphe (représentation mémoire).

```
// Représentation du graphe
void rep_graph(int num_sommets) {
    for (int i = 0; i < num_sommets; i++) {
        for (int j = 0; j < num_sommets; j++) {
            printf("%d ", matrice_adj[i][j]);
        }
        printf("\n");
    }
}
```

Complexité :

La complexité est également $O(n^2)$ où n est le nombre de sommets dans le graphe.

3. Calculer la densité du graphe

```
// Densité
double densite(num_sommets) {
    int num_arcs = 0;
    for (int i = 0; i < num_sommets; i++) {
        for (int j = 0; j < num_sommets; j++) {
            if (matrice_adj[i][j]) {
                num_arcs++;
            }
        }
    }
    return (2.0 * num_arcs) / (num_sommets * (num_sommets - 1));
}
```

Complexité :

La complexité est $O(n^2)$.

4. Recherche d'un nœud a dans le graphe (afficher le nœud et ses liens)

```
// Rechercher un nœud dans le graph
void rech_sommet(int sommet) {
    printf("Sommet %d:\n", sommet);
    for (int i = 0; i < num_sommets; i++) {
        if (matrice_adj[sommet][i]) {
            printf("    vers sommet %d\n", i);
        }
    }
}
```

Complexité :

La complexité peut être $O(1)$ si on affiche pas ses liens. Mais ce n'est pas le cas donc elle est de $O(n)$ où n est le nombre de sommets dans le graphe. Elle parcourt une seule fois la ligne de du sommet correspondant et imprime tous les arcs sortants de ce sommet.

5. Recherche d'un chemin entre un nœud a et un nœud b

```
// Rechercher un chemin entre deux nœuds dans le graph
bool rech_chemin(int origine, int dest) {
    if (origine == dest) {
        return true;
    }

    // Marquer le nœud source comme visité
    matrice_adj[origine][origine] = 2;

    // Vérifier si il y a un chemin vers chaque nœud adjacent
    for (int i = 0; i < num_sommets; i++) {
        if (matrice_adj[origine][i] == 1 && matrice_adj[i][i] != 2) {
            if (rech_chemin(i, dest)) {
                return true;
            }
        }
    }

    return false;
}
```

Complexité :

La complexité de cette fonction est $O(n^2)$ où n est le nombre de sommets dans le graphe car elle utilise une méthode de recherche en profondeur pour parcourir tous les sommets du graph à partir de l'origine et vérifie s'il y a un chemin vers la destination en visitant tous les sommets adjacents à chaque fois.

8. Ajouter un nœud a avec ses liens

```
// Ajouter un nœud avec ses liens au graph
void add_node_with_links(int node, int num_links, int links[]) {
    for (int i = 0; i < num_links; i++) {
        matrice_adj[node][links[i]] = 1;
        matrice_adj[links[i]][node] = 1;
    }
}
```

Complexité :

Complexité $O(m)$ où m est le nombre de liens a ajouter. Elle parcourt chaque lien une fois pour l'ajouter à la matrice d'adjacence.

C'est $O(m*2)$ si le graphe n'est pas orienté.

9. Supprimer un nœud a avec ses liens

```
// Supprimer un nœud avec ses liens
void supp_sommet_avec_liens(int node) {
    // Supprimer les liens du nœud
    for (int i = 0; i < num_sommets; i++) {
        matrice_adj[node][i] = 0;
        matrice_adj[i][node] = 0;
    }
    // Supprimer une colonne de la matrice
    for (int i = 0; i < num_sommets; i++) {
        for (int j = node; j < num_sommets - 1; j++) {
            matrice_adj[i][j] = matrice_adj[i][j + 1];
        }
    }
    // Supprimer une ligne de la matrice
    for (int i = node; i < num_sommets - 1; i++) {
        for (int j = 0; j < num_sommets; j++) {
            matrice_adj[i][j] = matrice_adj[i + 1][j];
        }
    }
}
```

Complexité

10. Ajouter un lien (arc ou arête) entre deux nœuds existants :

La complexité de cette fonction est $O(n^2)$ car elle parcourt tous les éléments de la matrice d'adjacence pour supprimer les liens et les lignes/colonnes du nœud spécifié.

```
// Ajouter une arête au graph
void add_lien_NO(int origine, int dest) {
    matrice_adj[origine][dest] = 1;
    // Le cas ou le lien n'est pas oriente
    matrice_adj[dest][origine] = 1;
}

// Ajouter une arête au graph oriente
void add_lien_NO(int origine, int dest) {
    matrice_adj[origine][dest] = 1;
}
```

Complexité :

La complexité de cette fonction est de $O(1)$, car elle modifie simplement une seule case dans la matrice d'adjacence pour ajouter une arête.

6. Evaluation expérimentale

LISTE VS MATRICE TEMPS EN MICROSECONDES

Taille n	Construction (Tps)	Affichage (Tps)	Recherche d'un nœud (Tps)
10	3 ms [LISTE] 1 ms [MATRICE]	15 ms [LISTE] 25 ms [MATRICE]	13 ms [LISTE] 24 ms [MATRICE]
20	/	/	/
30	/	/	/
50	20 ms [LISTE] 48 ms [MATRICE]	564 ms [LISTE] 542 ms [MATRICE]	17ms [LISTE] 12 ms [MATRICE]
<i>Recherche d'un chemin entre a et b</i>	<i>Ajouter un nœud a avec ses liens</i>	<i>Supprimer un nœud a avec ses liens</i>	<i>Ajouter un lien entre deux nœuds existants</i>
10 ms [LISTE] 39 ms [MATRICE]	4 ms [LISTE] 9 ms [MATRICE]	4 ms [LISTE] 4 ms [MATRICE]	3 ms [LISTE] 10 ms [MATRICE]
/	/	/	/
/	/	/	/
10 ms [LISTE] 22 ms [MATRICE]	12 ms [LISTE] 4 ms [MATRICE]	10 ms [LISTE] 6 ms [MATRICE]	4 ms [LISTE] 1 ms [MATRICE]

7. Conclusion

La principale différence entre l'utilisation d'une matrice d'adjacence et d'une liste d'adjacence pour représenter un graphe est la quantité d'espace nécessaire et la complexité temporelle de certaines opérations.

Dans une matrice d'adjacence de taille $V \times V$, où V est le nombre de sommets dans le graphe. Il est facile de vérifier s'il existe une arête entre deux sommets, la complexité est $O(1)$ car c'est juste une recherche dans une matrice. Mais la complexité d'espace est $O(V^2)$ car il a besoin de stocker toutes les arêtes possibles, même si elles n'existent pas.

D'autre part, une liste d'adjacence représente chaque sommet sous la forme d'une liste chaînée de ses sommets adjacents. La complexité d'espace est $O(V+E)$ où E est le nombre d'arêtes dans le graphe. C'est plus efficace en termes d'espace lorsque le graphe est clairsemé (contient peu d'arêtes). Mais la complexité temporelle de la vérification s'il existe une arête entre deux sommets est $O(V)$ car il faut parcourir la liste pour vérifier si l'arête existe.

En conclusion, la matrice d'adjacence est préférable lorsque le graphe est dense et que le nombre d'arêtes est proche de V^2 , tandis que la liste d'adjacence est plus efficace en termes d'espace et préférable lorsque le graphe est clairsemé et lorsque le graphe est dynamique, c'est-à-dire que le graphe change fréquemment, avec des sommets et des arêtes qui sont ajoutés ou supprimés. C'est parce qu'ajouter ou supprimer une arête ou un sommet dans une matrice d'adjacence nécessite de décaler un grand nombre d'éléments, ce qui peut être long.

Annexe : Des captures d'écran illustrant l'exécution du programme

I. LISTE D'ADJACENCE

LISTE D'ADJACENCE

11. Construction d'un graphe non orienté de 5 sommets.
12. Construction d'un graphe orienté de 5 sommets.
13. Construction d'un graphe de 50 sommets.
2. Affichage du graphe.
3. Calculer la densité du graphe.
4. Recherche d'un nœud a dans le graphe (afficher le nœud et ses liens).
5. Recherche d'un chemin entre un nœud a et un nœud b.
6. Recherche du chemin le plus court entre deux nœuds a et b.
7. Recherche d'une composante (fortement) connexe à partir d'un nœud a.
8. Ajouter un nœud a avec ses liens.
9. Supprimer un nœud a avec ses liens.
10. Ajouter un lien (arc ou arête) entre deux nœuds existants.

```
11
Construction terminee
Temps d'exécution : 4.000000 micro-secondes

Attends 2s
```

```
2
0 : 1 2 3
1 : 0 2
2 : 0 1
3 : 0
Temps d'exécution : 17.000000 micro-secondes

Attends 2s
```

```
3
Densité : 1.333333
Temps d'exécution : 17.000000 micro-secondes

Attends 2s
```

```
4
Entre le sommet
2
Sommet trouvé.
Temps d'exécution : 17.000000 micro-secondes

Attends 2s
```

```
6
Entre le noeud a
1
Entre le noeud b
3
Le plus court chemin entre 1 et 3 a une longueur de 2: 1 0 3 Temps d'exécution : 5.000000 micro-secondes
Attends 2s
```

```
8
Ecrit le noeud a (Choisi un noeud qui n'existe pas)
23
Combien de liens ?
1
Ecrit lien 1
3
Ecrit 1 si c'est un lien orienté
Ecrit 0 sinon.
0
```

```
10
Pour ajouter un lien entre deux noeuds :
Ecrit 1 si c'est un lien orienté
Ecrit 0 sinon.
1
Choisi le noeud a
3
Choisi le noeud b
1
Temps d'exécution : 9.000000 micro-secondes
Attends 2s
```

```
2
0 : 1 2 3
1 : 0 2
2 : 0 1
3 : 0 1
Temps d'exécution : 22.000000 micro-secondes
Attends 2s
```

II. MATRICE D'ADJACENCE

MATRICE D'ADJACENCE

11. Construction d'un graphe non orienté de 5 sommets.
12. Construction d'un graphe orienté de 5 sommets.
13. Construction d'un graphe de 50 sommets.
2. Affichage du graphe.
3. Calculer la densité du graphe.
4. Recherche d'un nœud a dans le graphe (afficher le nœud et ses liens).
5. Recherche d'un chemin entre un nœud a et un nœud b.
6. Recherche du chemin le plus court entre deux nœuds a et b.
7. Recherche d'une composante (fortement) connexe à partir d'un nœud a.
8. Ajouter un nœud a avec ses liens.
9. Supprimer un nœud a avec ses liens.
10. Ajouter un lien (arc ou arête) entre deux nœuds existants.

□

```
12
Construction terminee
Temps d'exécution : 2.000000 micro-secondes

Attends 2s
```

```
2
0 1 1 1 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
Temps d'exécution : 34.000000 micro-secondes

Attends 2s
```

```
3
0.400000
Temps d'exécution : 27.000000 micro-secondes

Attends 2s
```

```
4
Entre le sommet
3
Sommet 3:
Temps d'exécution : 14.000000 micro-secondes

Attends 2s
```

```
8
Ecrit le noeud a (Choisi un noeud qui n'existe pas)
6
Combien de liens ?
2
Ecrit lien 1
1
Ecrit 1 si c'est un lien orienté
Ecrit 0 sinon.
0
Temps d'exécution : 12.000000 micro-secondes
Ecrit lien 2
2
Ecrit 1 si c'est un lien orienté
Ecrit 0 sinon.
1
Temps d'exécution : 3.000000 micro-secondes

Attends 2s
```

```
9
Ecrit le noeud a supprimer
0
Temps d'exécution : 3.000000 micro-secondes

Attends 2s
```

```
2
0 1 0 0
0 0 0 0
0 0 0 0
0 0 0 0
Temps d'exécution : 35.000000 micro-secondes

Attends 2s
```